

Way2Quran Architecture

A Beginner's Tour for New Engineers

Goal of this deck: by the end you'll be able to draw the whole system on a whiteboard, know where any piece of code lives, and trace a single click from the browser all the way to the database and back.

 Each slide is one idea. Diagrams render on GitHub and in any Mermaid viewer. Speaker notes (the "say this out loud" bits) are in the `> quote` blocks.

How to read this deck

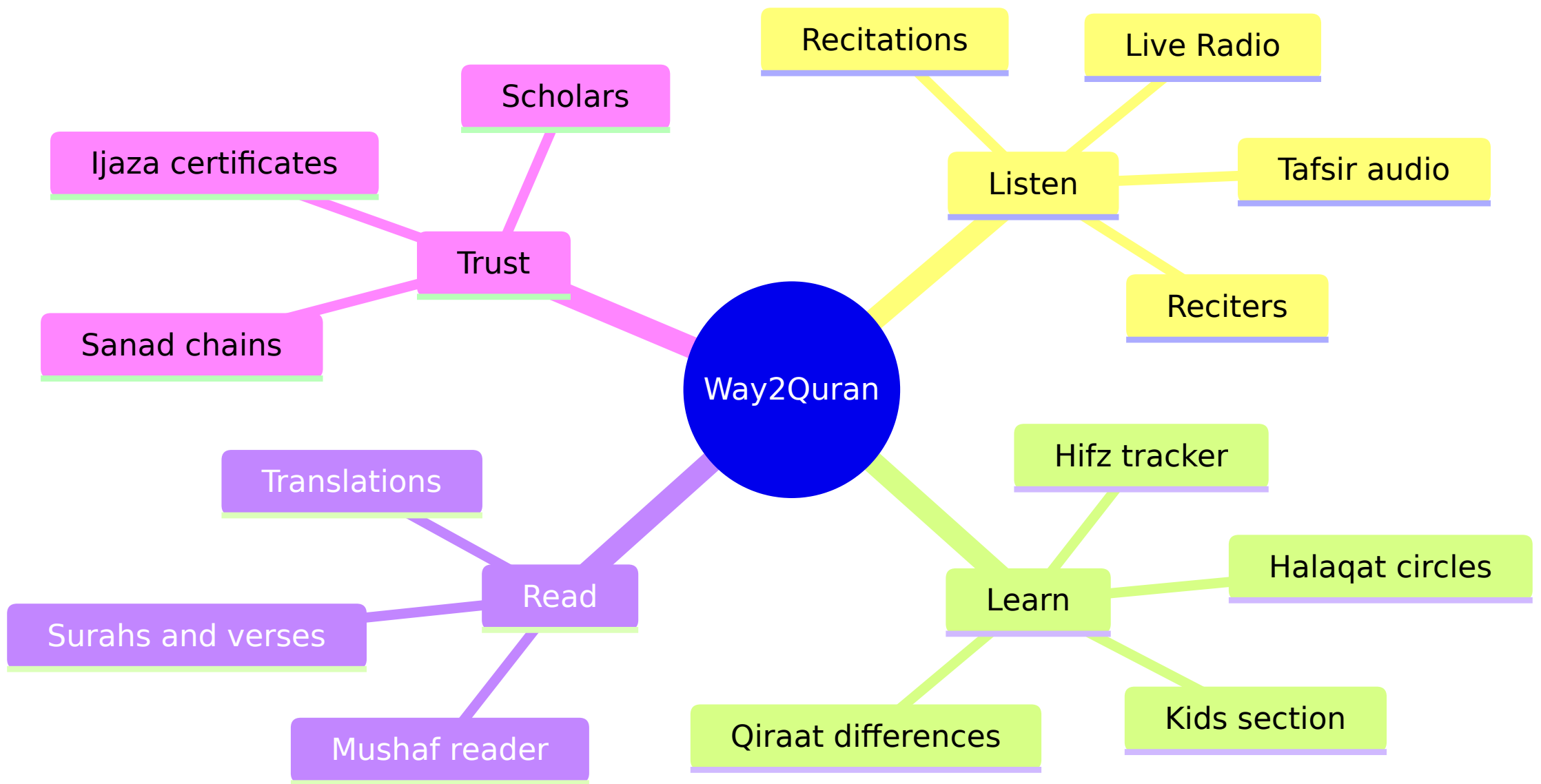
Symbol	Meaning
	A concept or building block
	Where it lives in the repo
	A flow / sequence to trace
	A "why it's done this way" insight
	Glossary term (defined at the end)

If a word is new to you, check the **Glossary** on the last slides — every  term is defined there.

Slide 1 — What is Way2Quran, in one sentence?

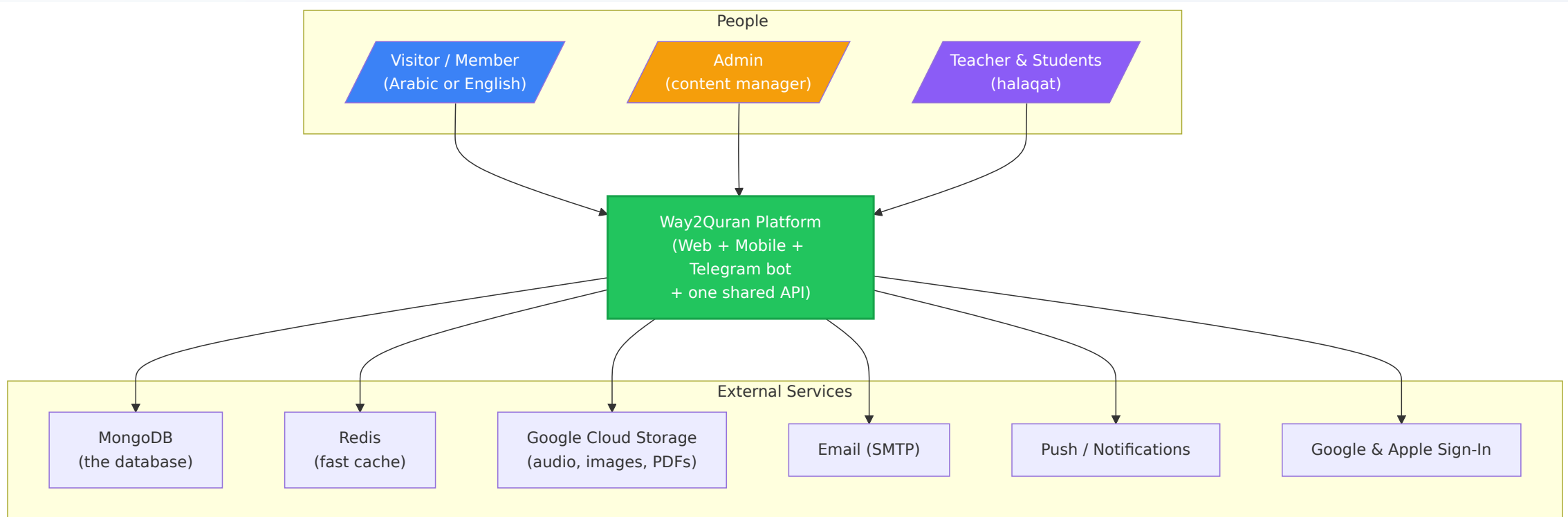
A bilingual (Arabic-first, RTL) website + mobile app where people **browse Quran reciters, listen to recitations, download audio, memorize (hifz), join study circles (halaqat), and verify chains of transmission (sanad)** — backed by one shared API.

✚ It is **not** a single program. It is a **platform**: several front-ends talking to one back-end brain.



Slide 2 — The 10,000-foot view (who talks to what)

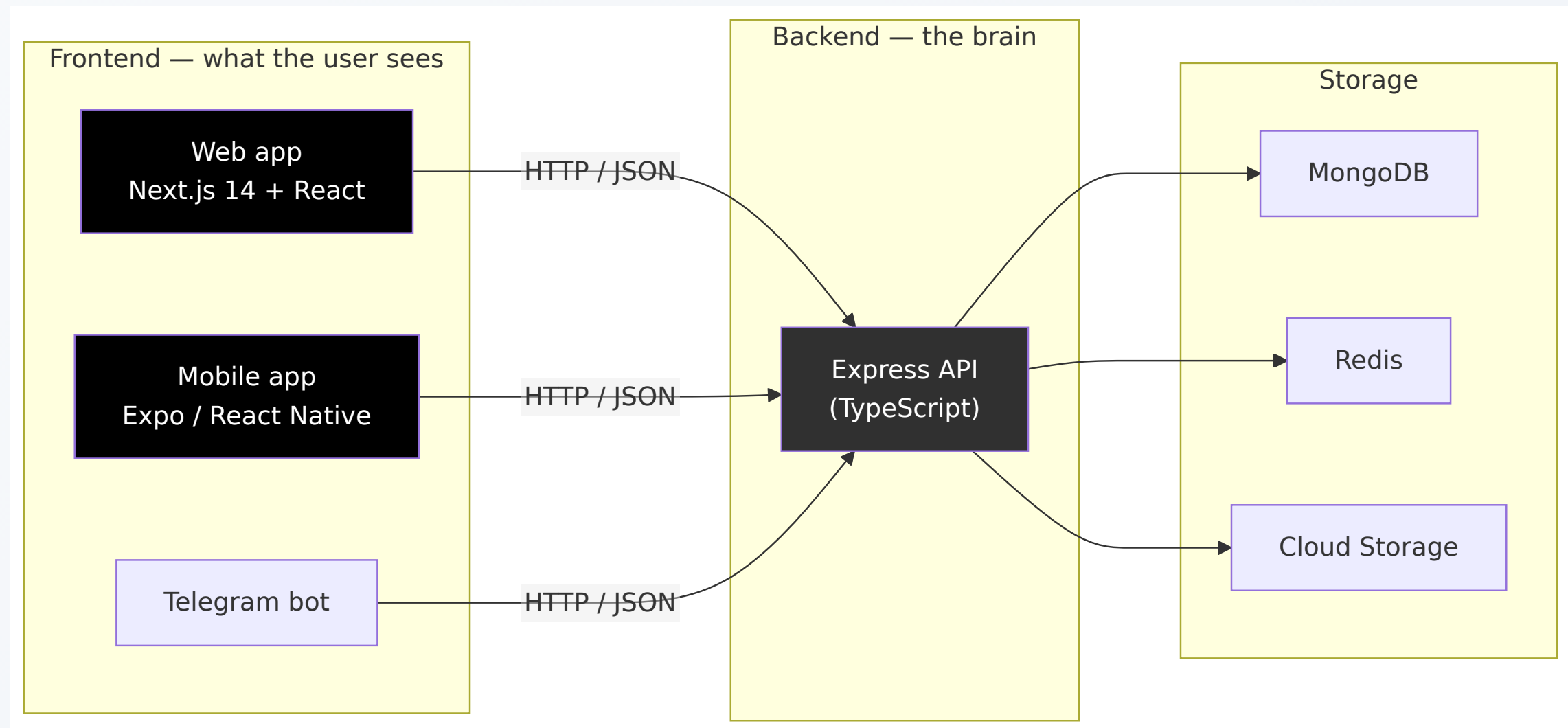
This is a **C4 Level 1 (System Context)** diagram: the system as one box, plus who/what touches it.



💡 Notice the system does **not** store audio files itself. Big files live in Google Cloud Storage. The database only stores *links* to them. This keeps the database small and fast.

Slide 3 — The two halves: Frontend and Backend

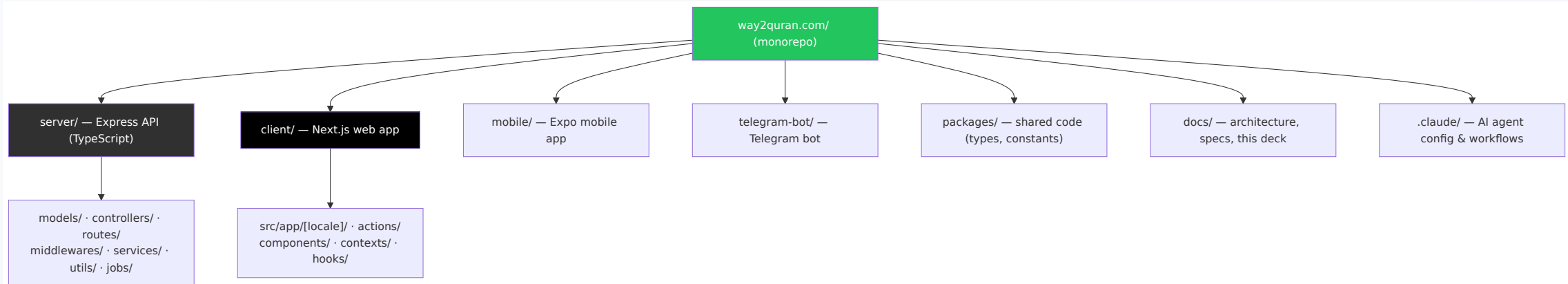
Almost every web app is split into a **frontend** (what you see) and a **backend** (the logic + data). Way2Quran is no different.



💡 **One backend, many front-ends.** The web app, the mobile app, and the Telegram bot all call the *same* API. Build a feature in the API once → every client can use it.

 client/ = web · mobile/ = app · telegram-bot/ = bot · server/ = the API

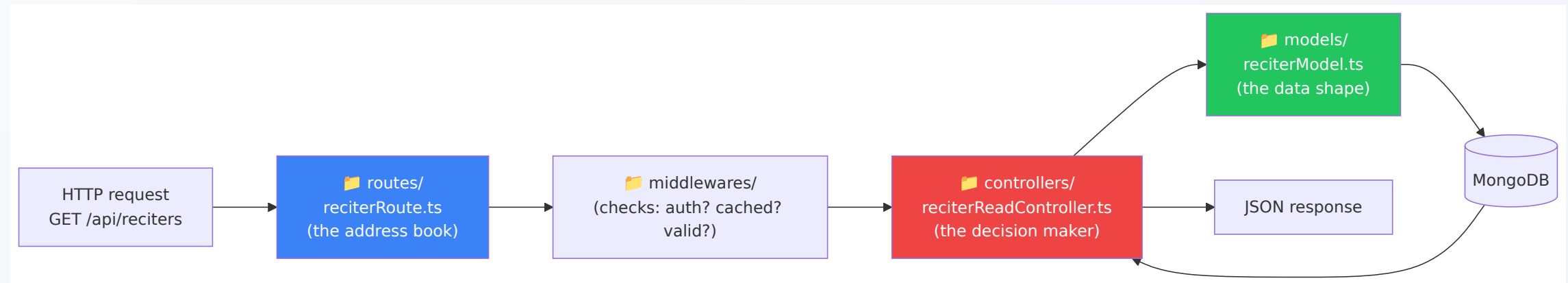
Slide 4 — The repo at a glance



💡 A **monorepo** 📖 means all these apps live in one Git repository. They can share code (the `packages/` folder) and ship together.

Slide 5 — The backend pattern: Route → Controller → Model

This is the single most important pattern in the backend. Learn it once and every one of the ~60 features makes sense.

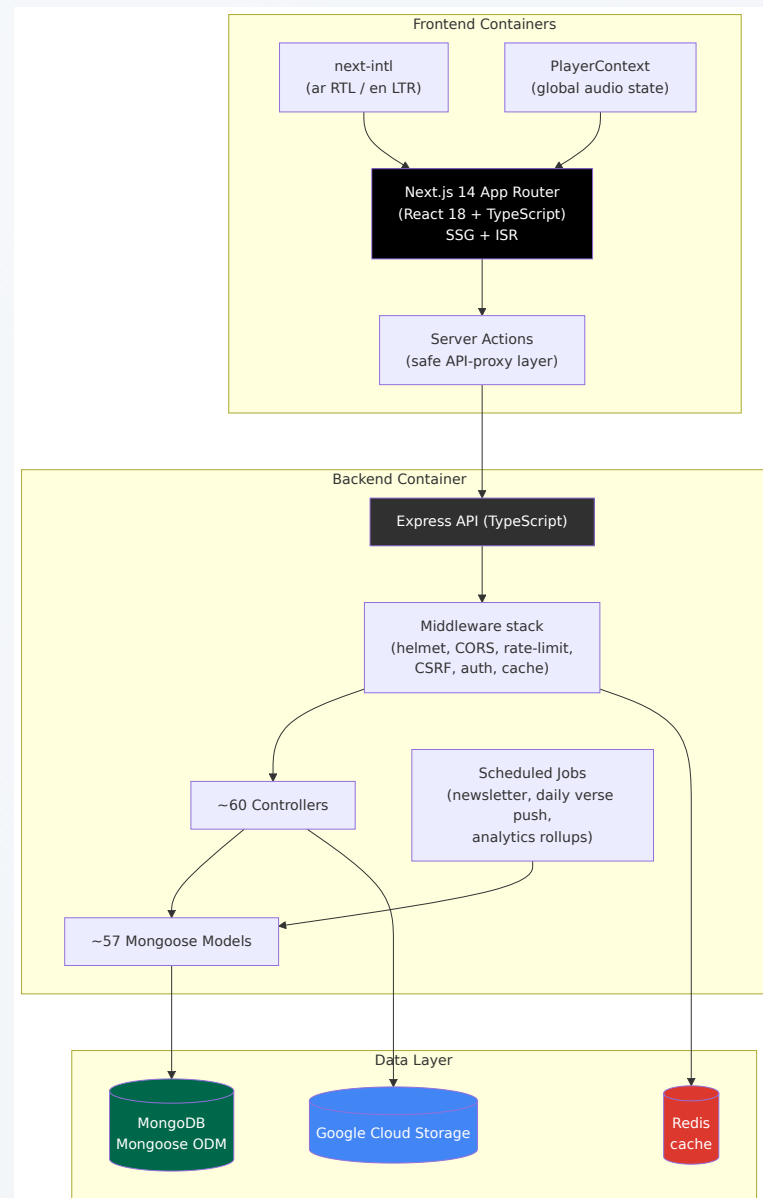


Layer	Job	Analogy
Route	Maps a URL to a function	The restaurant menu
Middleware 📖	Runs <i>before</i> the controller (auth, cache, validation)	The bouncer + the host
Controller	Does the actual work, decides the response	The chef
Model 📖	Defines the shape of data & talks to the DB	The recipe + pantry

💡 When you add a feature, you almost always touch these same 3-4 files. Find an existing feature (e.g. `bookmark`) and copy its shape.

Slide 6 — Container view (C4 Level 2)

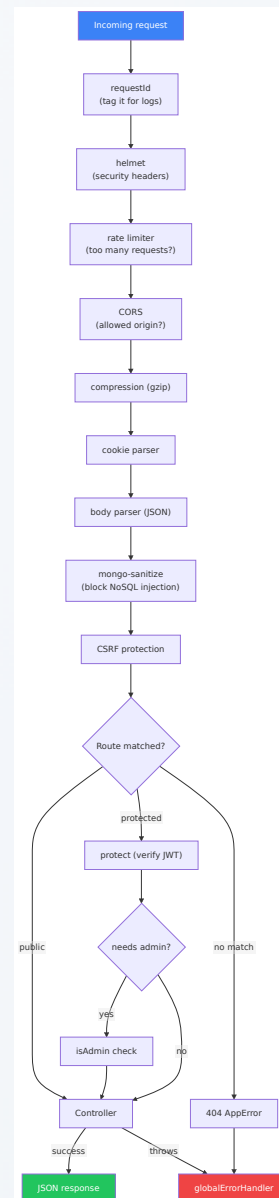
Zooming into the platform box from Slide 2. These are the **deployable pieces** and the tech in each.



💡 **Server Actions** 📖 are a Next.js feature. The browser never calls the Express API directly — it calls a server-side function, which calls the API. This hides API URLs and keeps secrets on the server.

Slide 7 — How a request actually flows through the backend

Every request runs a gauntlet of middleware *before* reaching your controller. Order matters.

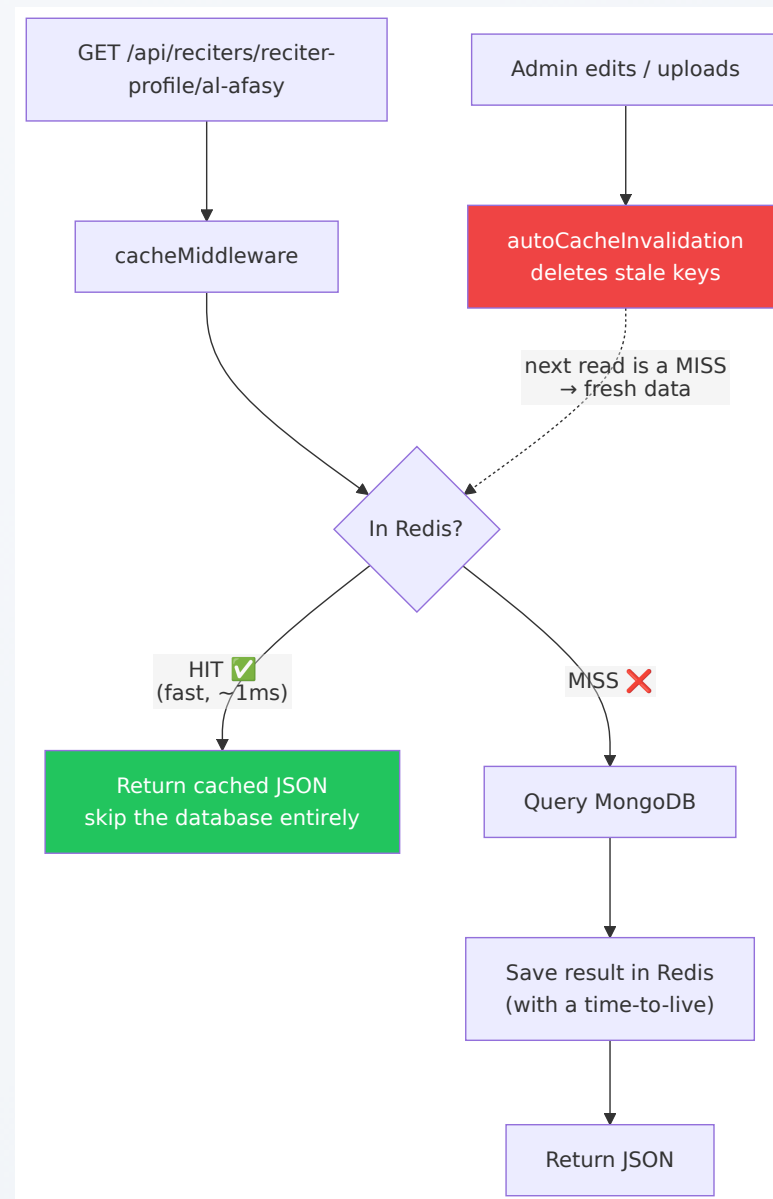


💡 All errors funnel into **one** place: `globalErrorHandler` in `errorController.ts`. Controllers just `throw new AppError(...)` and forget about formatting. In dev you get a stack trace; in prod, a safe message.

📁 The whole chain is wired up in `server/app.ts` .

Slide 8 — Caching: why the site is fast (Redis)

The database is the source of truth, but querying it for every visitor is slow and expensive. So we put a **cache** 📖 in front of it.

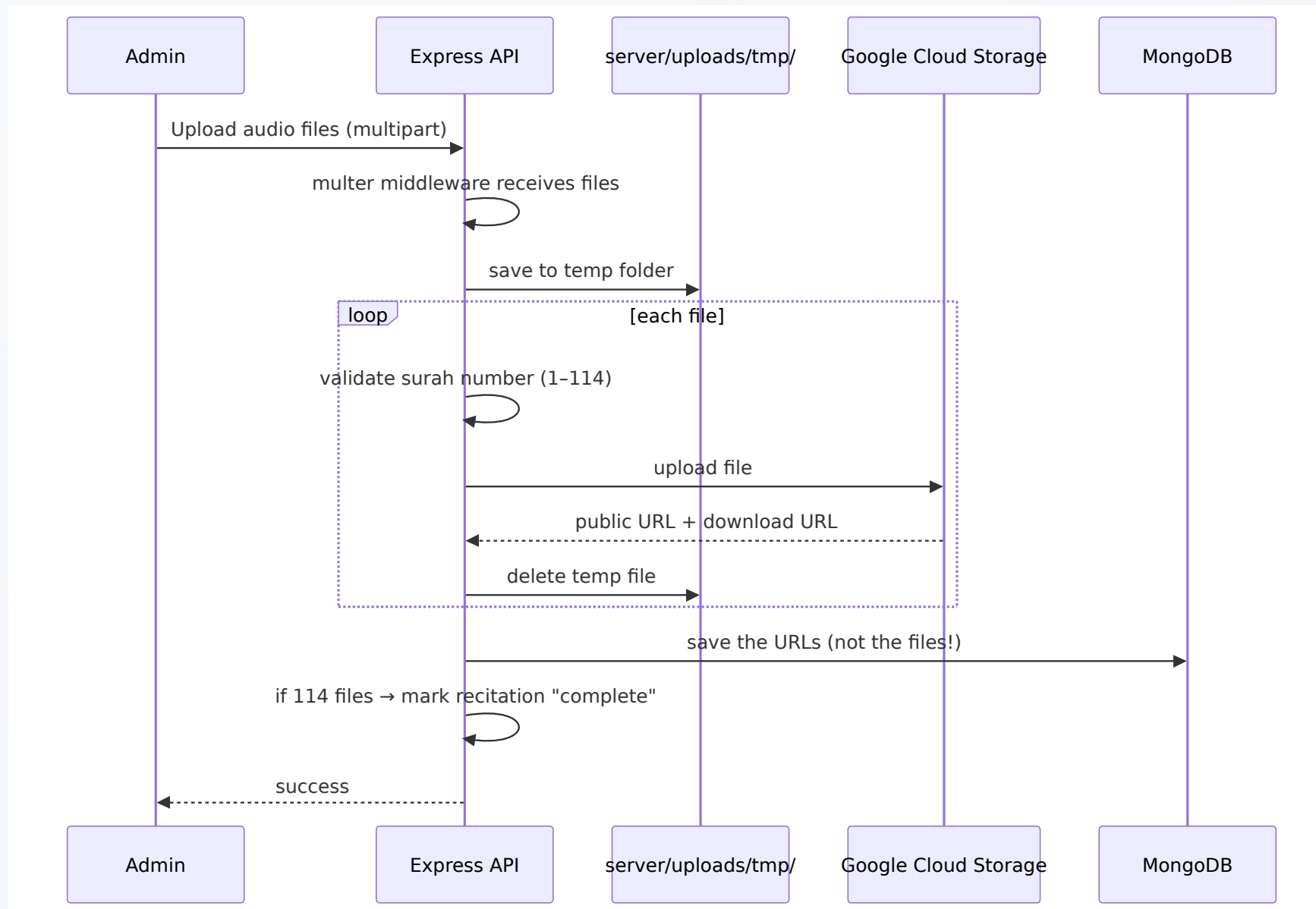


💡 The tricky part of caching is **invalidation** — throwing away stale data. After any successful write (POST/PUT/PATCH/DELETE), `autoCacheInvalidation` clears the affected keys so the next read fetches fresh

data. "There are only two hard things in computer science: cache invalidation and naming things." 😊

Slide 9 — Where the big files live (Cloud Storage)

Audio (MP3s), reciter photos, ZIP downloads, and Mushaf PDFs are **large**. They don't belong in a database.

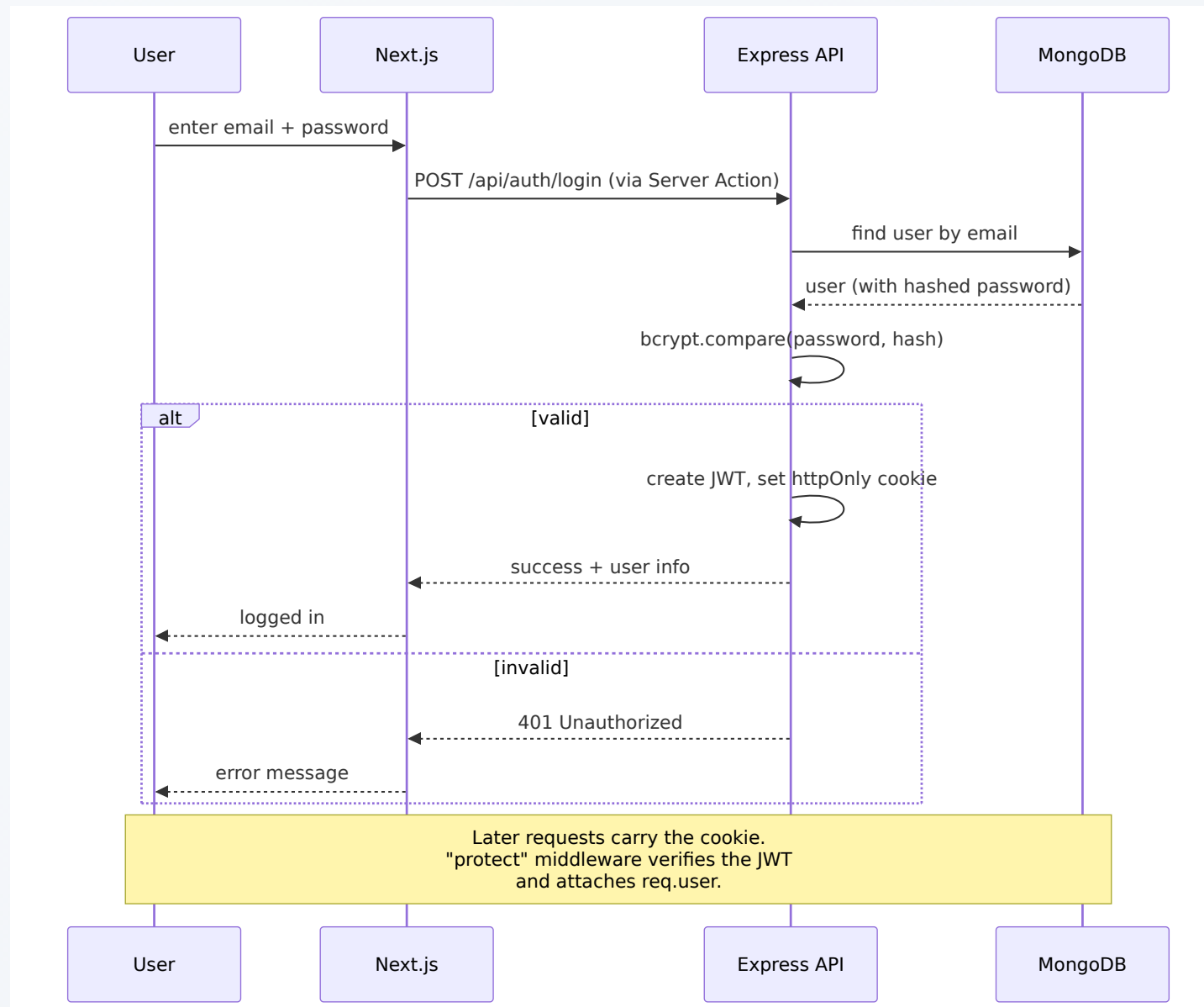


💡 The pattern: **files** → **cloud**, **links** → **database**. The browser later streams audio *directly* from Cloud Storage's CDN, not through our API. Our server stays light.

 server/utils/ has uploadFileToGCS / deleteFileFromGCS .

Slide 10 — Authentication: how login works

Way2Quran supports email/password, Google, and Apple sign-in, plus optional two-factor (2FA) for admins.



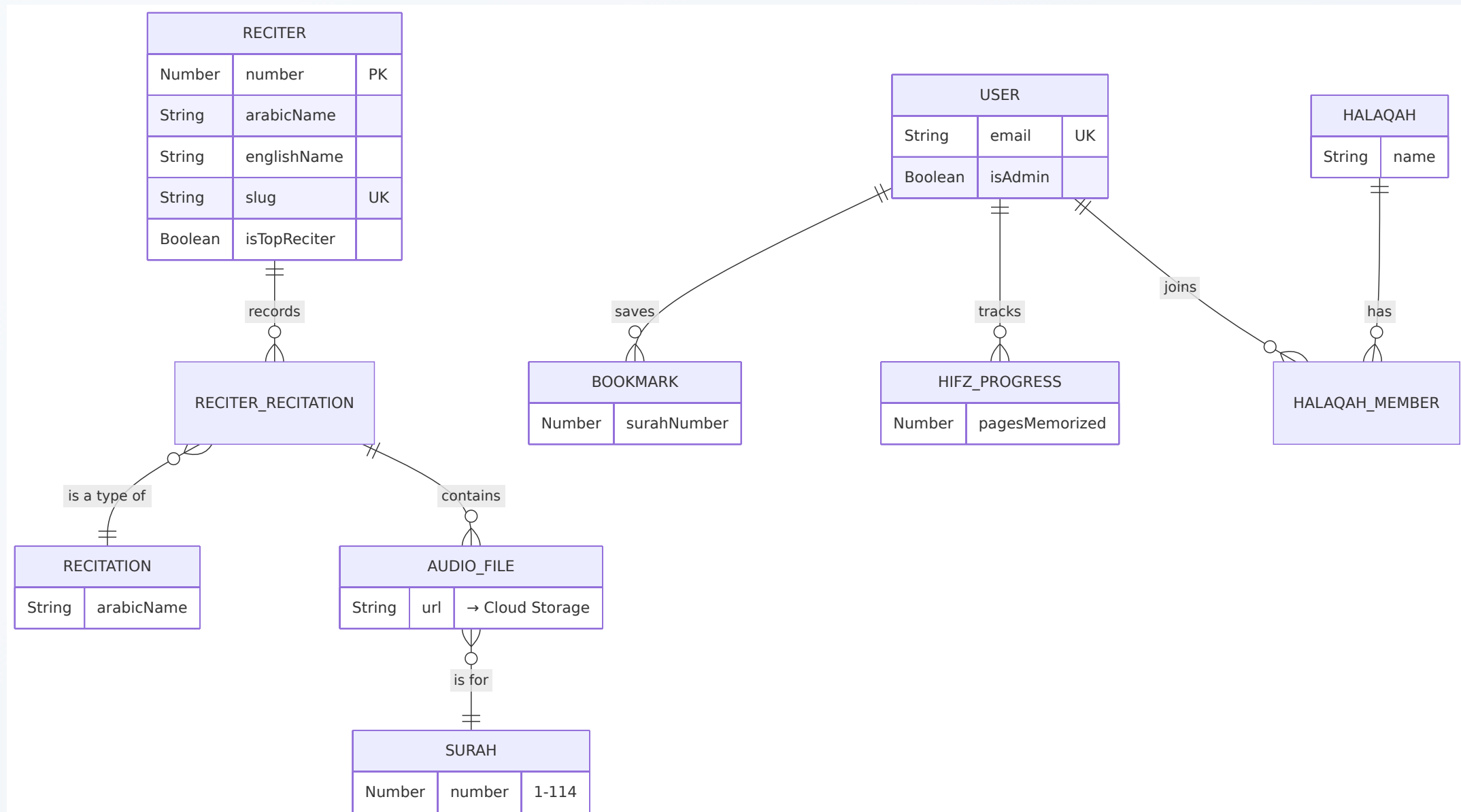
💡 Three security ideas to remember:

1. Passwords are **hashed** 📖 with bcrypt — never stored in plain text.

2. The login token (a **JWT** 📖) is stored in an **httpOnly cookie** — JavaScript can't read it, which blocks a whole class of attacks.
3. `protect` = "must be logged in"; `isAdmin` = "must be an admin". Routes pick what they need.

Slide 11 — The data model (simplified)

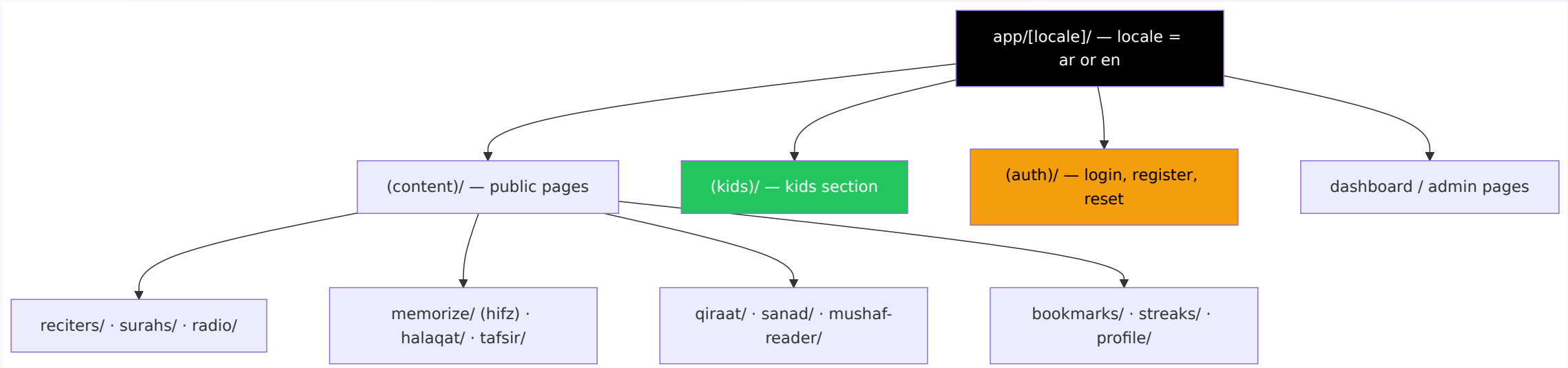
The core relationship: a **Reciter** records many **Recitations**; each recitation is a set of **audio files**, one per **Surah**.



💡 There are **~57 models** in total (sanad chains, tafsir, qiraat, playlists, analytics, etc.). You don't need to memorize them. Learn this core, and the rest follow the same Mongoose pattern. 📁 `server/models/`

Slide 12 — The frontend: Next.js App Router

The web app uses Next.js's **App Router**: folders = URLs. The `[locale]` segment makes every page bilingual.

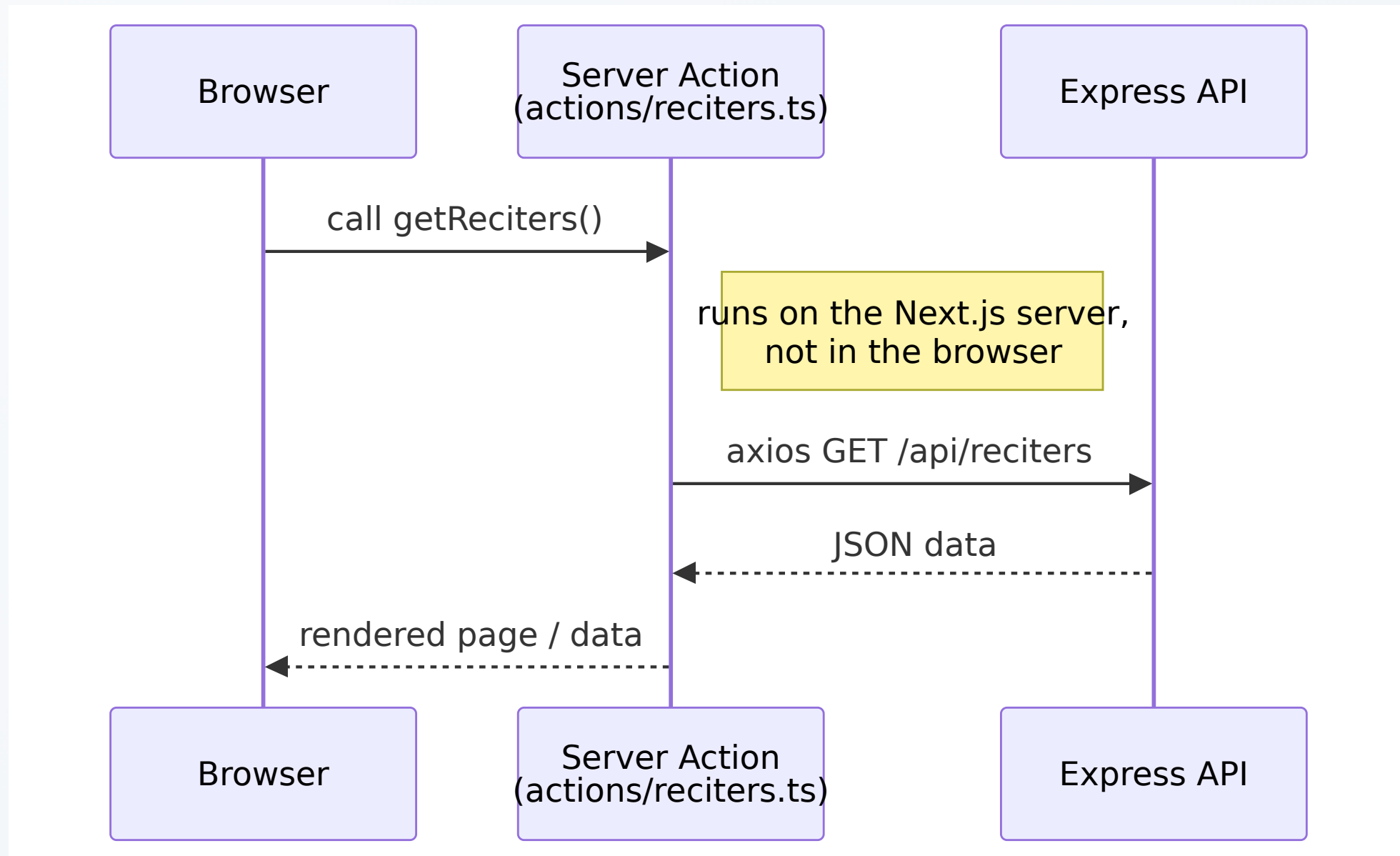


💡 The `(content)`, `(kids)`, `(auth)` parentheses are **route groups** 📖 — they organize files and let each group have its own layout, *without* adding to the URL. `/ar/reciters` not `/ar/(content)/reciters`.

📁 `client/src/app/[locale]/`

Slide 13 — How the frontend gets data: Server Actions

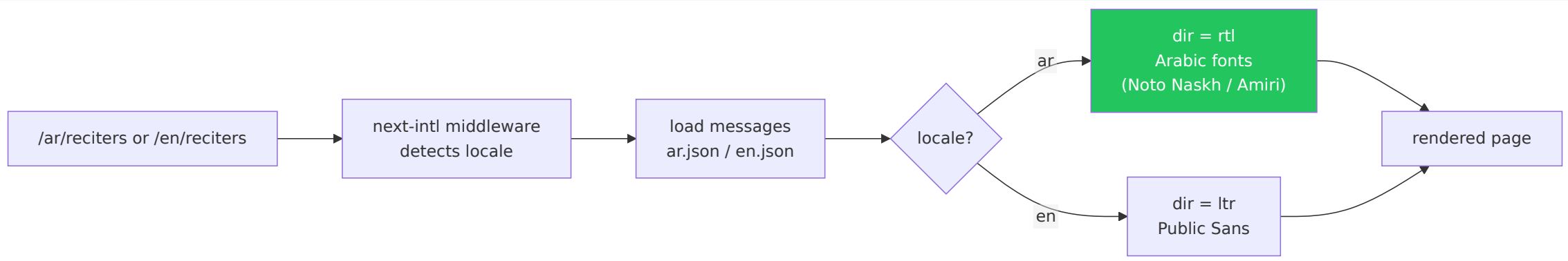
The browser never holds an API URL or a secret. It calls a server function.



💡 **Why bother?** Secrets (tokens, internal URLs) stay on the server. The browser only ever sees the final HTML/data. This is the "BFF" (backend-for-frontend) pattern.

Slide 14 — Internationalization (i18n) & RTL

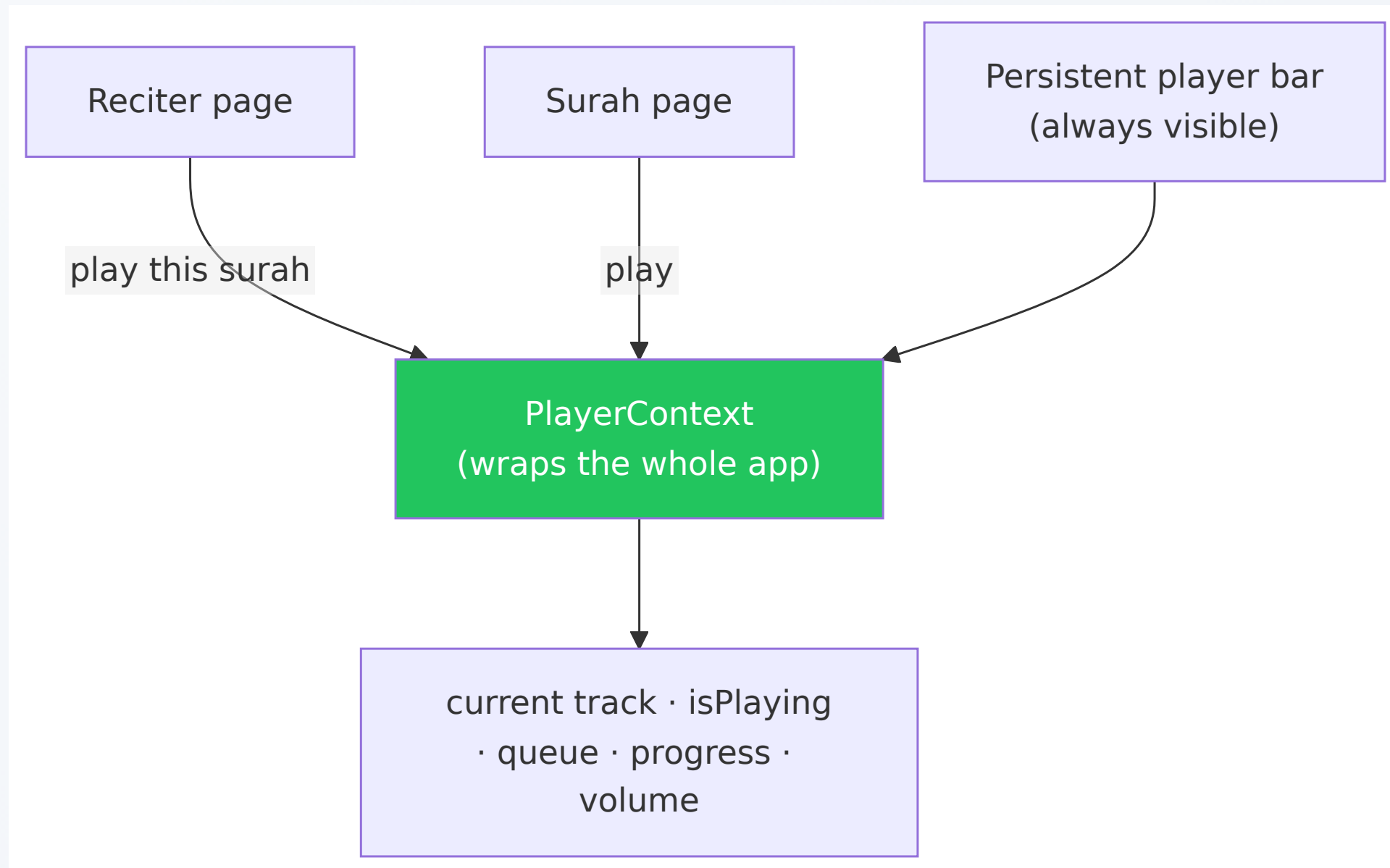
Arabic is the **default** language and reads **right-to-left**. This is a first-class concern, not an afterthought.



💡 Components never hard-code text. They call `useTranslations()` and look up a **key**. Add a string to *both* `ar.json` and `en.json` or the build breaks. 📁 `client/src/public/locales/`

Slide 15 — The audio player (the one piece of global state)

Most state in React is local to a page. But the audio player must keep playing as you navigate. That needs **global state**

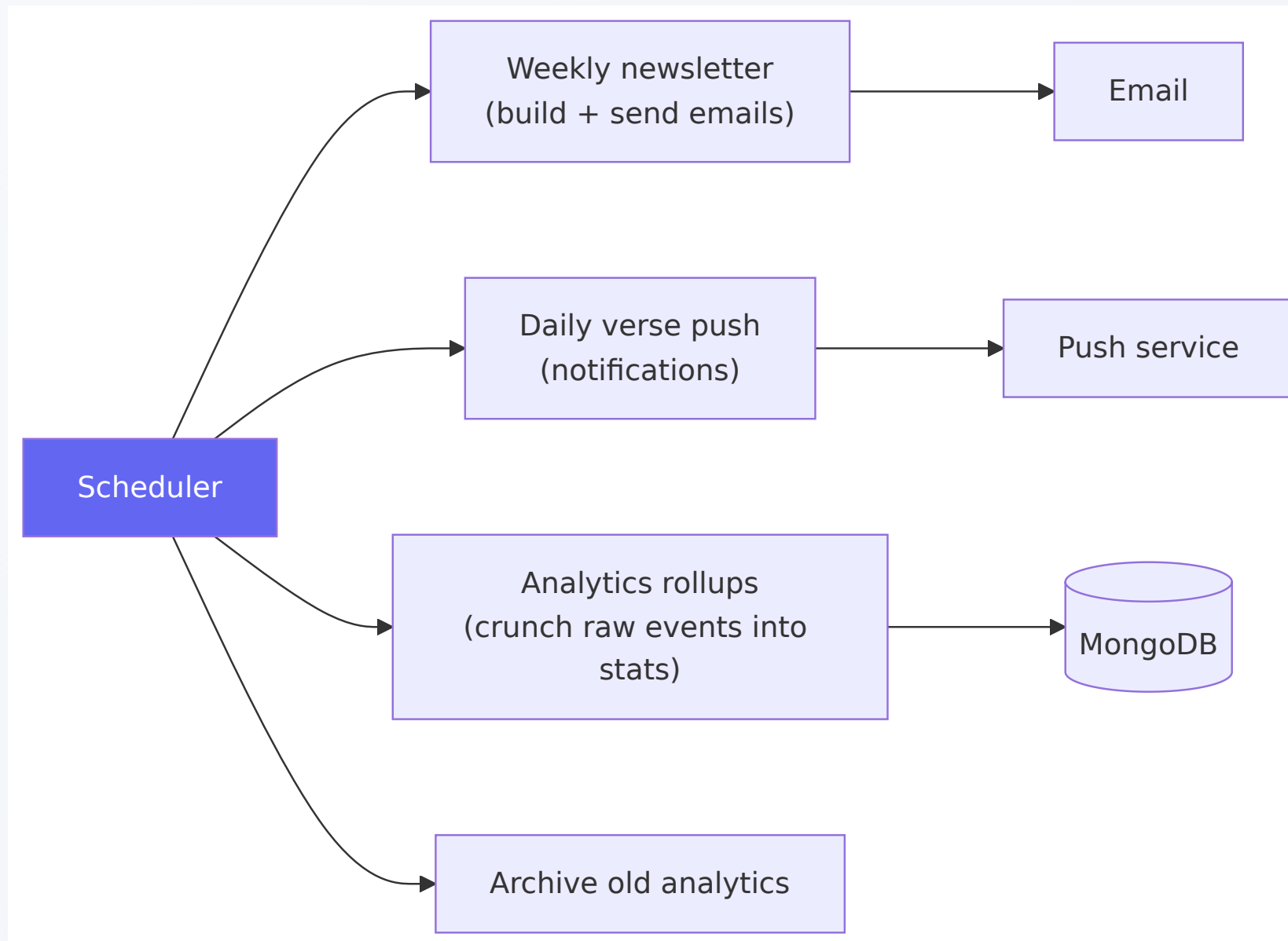


💡 Way2Quran deliberately keeps global state **tiny** — only the audio player uses a React Context. Everything else is

fetches per-page. Less global state = fewer bugs. 📁 `client/src/contexts/PlayerContext`.

Slide 16 — Background jobs (work that runs on a schedule)

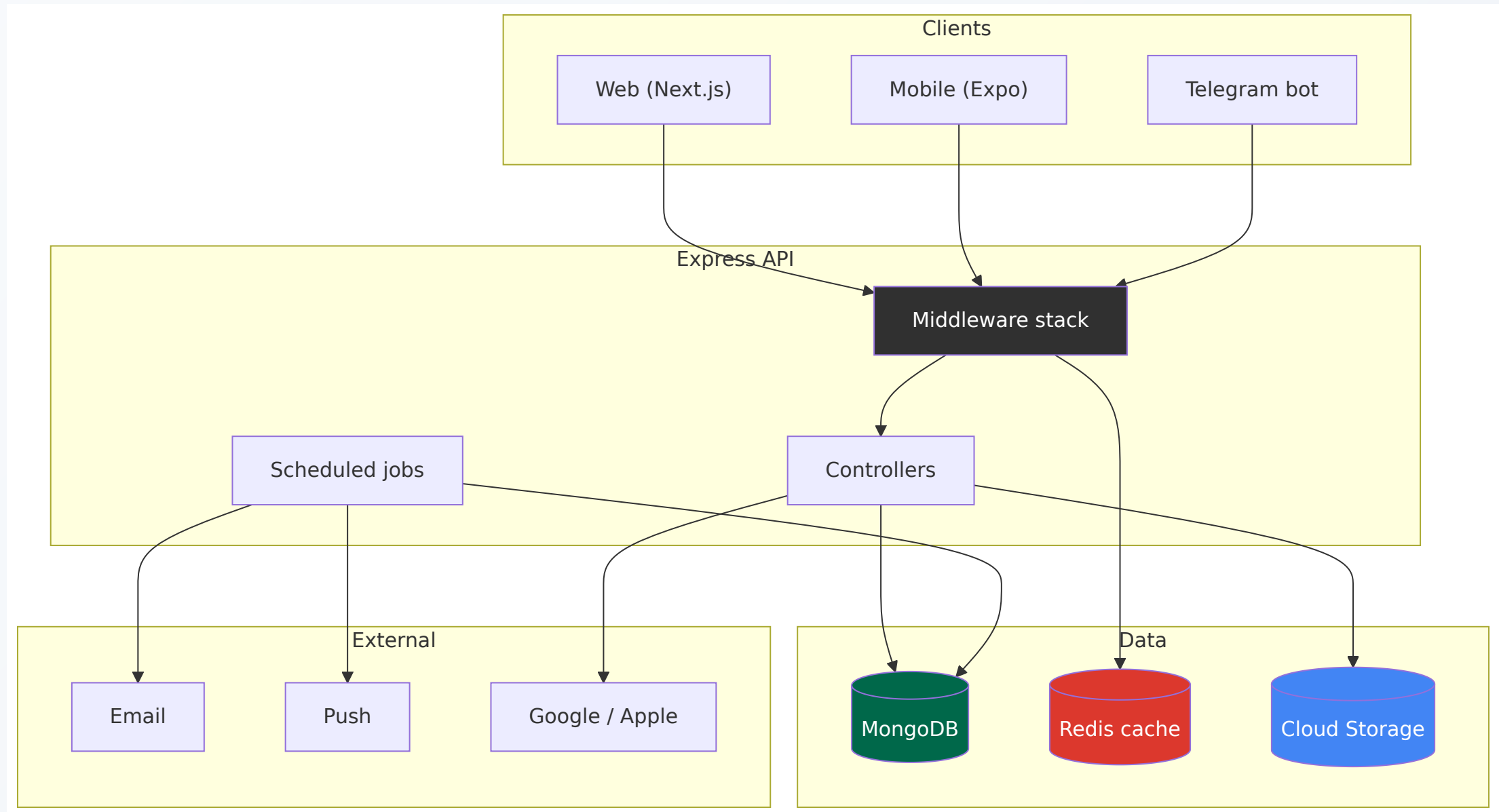
Some work shouldn't happen during a user's request — it runs on a timer in the background.



💡 Rule of thumb: if it's slow, periodic, or doesn't need a user waiting, it's a **job**, not a request handler. 📁

`server/jobs/` and `server/services/`.

Slide 17 — The full picture (put it all together)



🎯 If you can explain this diagram out loud, you understand Way2Quran's architecture.

Slide 18 — Your first day: a checklist

1. 📁 **Read** `CLAUDE.md` at the repo root — conventions, commands, git rules.
2. ▶ **Run it:** `npm run setup` then `npm run dev` (starts API + web together).
3. 🔍 **Trace one feature end-to-end.** Pick `bookmarks`:
 - `client/src/app/[locale]/(content)/bookmarks/` (the page)
 - `client/src/actions/` (the server action)
 - `server/routes/bookmarkRoute.ts` → `controllers/bookmarkController.ts` → `models/bookmarkModel.ts`
4. 🧪 **Find its tests:** `server/__tests__/bookmarkController.test.ts`.
5. ✏ **Make a tiny change** (e.g. a label) and watch it hot-reload.

💡 Tracing one existing feature top-to-bottom teaches more than reading ten slides. The pattern repeats everywhere.

Slide 19 — Tech stack cheat sheet

Layer	Technology	What it does
Web frontend	Next.js 14 + React 18	Pages, SSR/SSG, routing
Mobile	Expo / React Native	iOS + Android app
Styling	Tailwind CSS	Utility-first CSS
i18n	next-intl	Arabic (RTL) + English
Backend	Express + TypeScript	REST API
Database	MongoDB + Mongoose	Document store
Cache	Redis	Fast response cache
File storage	Google Cloud Storage	Audio, images, PDFs
Auth	JWT + bcrypt + OAuth + 2FA	Login & sessions
Email	Nodemailer (SMTP)	Newsletters, verification
Tests	Jest + Playwright	Unit + E2E

Glossary 📖 (1/2)

- **Monorepo** — one Git repository holding several apps (web, mobile, API) that can share code.
- **API** — Application Programming Interface; the backend's set of URLs that return data (here, JSON over HTTP).
- **Route** — a mapping from a URL (e.g. `/api/reciters`) to the code that handles it.
- **Controller** — the function that does the work for a route and decides the response.
- **Model** — a definition of a data shape (a "schema") that also reads/writes the database (via Mongoose).
- **Middleware** — code that runs *before* a controller, in a chain (auth checks, caching, validation, logging).
- **ODM (Mongoose)** — Object-Document Mapper; lets you work with MongoDB documents as JavaScript objects.

Glossary 📖 (2/2)

- **Cache (Redis)** — a fast, temporary store of recent answers so we don't hit the database every time.
- **Cache invalidation** — deleting stale cached data after something changes.
- **JWT** — JSON Web Token; a signed string proving you're logged in, stored in an httpOnly cookie.
- **Hashing (bcrypt)** — a one-way scramble of passwords; we store the hash, never the password.
- **Server Action** — a Next.js server-side function the browser calls instead of hitting the API directly.
- **Route group** — Next.js folders in `(parentheses)` that organize files/layouts without changing the URL.
- **Global state / Context** — data shared across the whole app (here, only the audio player).
- **SSG / ISR** — Static Site Generation / Incremental Static Regeneration; pre-building pages for speed.
- **CDN** — Content Delivery Network; serves big files (audio) fast from servers near the user.

Questions?

 Go deeper:

- `docs/architecture/SYSTEM-ARCHITECTURE.md` — full C4 diagrams
- `docs/architecture/DATA-FLOW.md` — detailed flows
- `docs/api/API-REFERENCE.md` — every endpoint
- `CLAUDE.md` — conventions & commands

The best next step: clone, `npm run dev`, and trace one feature. The architecture will click.